KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous, Affiliated to Anna University, Accredited by NAAC 'A' Grade)

Department of Artificial Intelligence and Data Science

CASE STUDY

E-commerce Sales Analysis: Exploratory Data Analysis, Statistical Visualization, and Deep Neural Network Prediction of Repeat Purchases

Course Code & Title	U21ADP05 – Exploratory Data Analysis and Visualization
Assignment	Assignment II
Academic Year & Semester	2026 – 2027 (ODD SEM) | Semester V
Batch	2024 · 2028
Course Coordinator	Ms. REVATHY M
Student Name	Santhosh G
Roll / Register Number	24AD102
Date of Submission	15 / 10 / 2026
Evaluation Rubrics	Total 40 Marks (CO3: 8 | CO4: 16 | CO5: 16 + Deep Learning Enrichment Component)
GitHub Repository	https://github.com/santhosh07072007-pixel

2. Abstract

Online retailers depend on repeat customers, yet identifying who will return is difficult. This study analyses a synthetic e-commerce dataset of 2,000 orders with 20 customer, product and delivery features, following Ben Fry's seven stages of data visualization. Preprocessing covers duplicate audit, Tukey IQR outlier quantification, leakage-free imputation and stratified 70/15/15 partitioning. Welch t-tests, Mann-Whitney U tests and Chi-Square tests show that Prime membership, faster shipping and prior order history are linked to repeat purchase. A multi-layer perceptron, tuned over three configurations with early stopping, reaches 68.33% test accuracy and ROC-AUC 0.6919, close to a logistic regression baseline (0.7112). Electronics yields 46.7% of revenue, and November and December contribute 25.8%.

(Word Count: 110 words — within the required 100–150 word specification.)

Data note. No real transaction log was supplied, so the analysis runs on a synthetic dataset generated with a fixed random seed (42) by the script in Section 11. Every number, table and figure in this report comes from that run. The conclusions describe the synthetic data and should not be read as findings about a real retailer.

3. Introduction & Objective

3.1 Business Background and Motivation

Acquiring a new online customer typically costs far more than retaining an existing one, so repeat purchase behaviour is a central metric for e-commerce planning. Revenue in online retail is also unevenly distributed: a small number of categories, seasons and large baskets usually account for a disproportionate share of sales. Understanding both patterns lets a retailer decide where to stock, when to discount and which customers to target with loyalty offers.

When a retailer cannot tell which customers are likely to return, it faces three recurring problems:

1. **Misallocated discounts:** blanket promotions erode margin on customers who would have returned anyway.

2. **Unplanned seasonal peaks:** demand spikes in festive months strain inventory and delivery capacity.

3. **Delivery-driven churn:** slow shipping lowers ratings and weakens the chance of a second order.

This case study turns these questions into a measurable pipeline: explore revenue structure, test which order and customer attributes differ between one-time and repeat buyers, and train a neural network to estimate the probability of a repeat purchase.

3.2 Technical Objectives

Step 1 — Dataset Selection: Generate a reproducible synthetic e-commerce orders dataset of 2,000 records and 20 features (15 original order, customer and delivery attributes plus 5 engineered features, strictly exceeding the ≥ 15 feature requirement).

Step 2 — Data Understanding & Preprocessing: Inspect missing values, verify duplicate independence, compute Tukey IQR outlier bounds, encode categorical variables, standardize numeric scales and enforce stratified 70/15/15 splits without data leakage.

Step 3 — Exploratory Data Visualization: Demonstrate Ben Fry's seven stages (Acquire, Parse, Filter, Mine, Represent, Refine, Interact) through figures covering univariate, categorical, bivariate, temporal and multivariate views.

Step 4 — Statistical & Distributional Exploration (CO4): Describe skewness and kurtosis, and run Welch t-tests, Mann-Whitney U tests, Chi-Square tests of independence and Cramér's V.

Step 5 — Deep Learning Neural Modeling (CO5 & Enrichment): Build, regularize and tune a multi-layer perceptron (MLP) across three hyperparameter configurations using scikit-learn, and evaluate convergence, generalization and permutation feature importance.

Step 6 & 7 — Model Evaluation & Synthesis: Produce the loss curve, accuracy curve, confusion matrix and ROC curve, benchmark against logistic regression, and summarize findings with references.

4. Dataset Description

The analysis uses a synthetic orders table of 2,000 rows. Each row is one order placed by a customer, described by demographic, product, payment, delivery and engagement attributes. The target variable **repeat_purchase** equals 1 if the customer placed a further order after this one and 0 otherwise. Category-level base prices, seasonal discount uplift in November and December, longer delivery in the Central region and Prime-member effects were built into the generator so that realistic patterns exist to be discovered; the generator code is shown in Section 11.

4.1 Feature Schema Dictionary

Table 1: Feature schema of the e-commerce orders dataset

Field Name	Data Type	Domain / Values	Category	Operational Description
order_month	Categorical (int)	1 to 12	Temporal	Calendar month of the order
customer_age	Continuous	18 to 65 years	Demographic	Age of the customer
gender	Categorical	F, M	Demographic	Customer gender
region	Categorical	North, South, East, West, Central	Geographic	Delivery region
category	Categorical	Electronics, Fashion, Home & Kitchen, Beauty, Books, Sports	Product	Product category
payment_method	Categorical	Credit Card, UPI, Debit Card, COD, Wallet	Transactional	Payment mode used
device	Categorical	Mobile, Desktop, Tablet	Behavioural	Device used to place the order
unit_price	Continuous	4.22 to 1093.11 currency units	Product	Price of one unit
quantity	Discrete	1 to 8 units	Transactional	Units in the order
discount_pct	Continuous	0.0 to 46.7 %	Transactional	Discount applied to the order
shipping_days	Discrete	1 to 16 days	Logistics	Days from order to delivery
is_prime	Binary Int	0 (No), 1 (Yes)	Customer	Prime membership indicator
prior_orders	Discrete	0 to 10 orders	Customer	Orders placed before this one
session_minutes	Continuous	1.0 to 57.6 min	Behavioural	Time spent on site before purchase
customer_rating	Continuous	1.1 to 5.0 stars	Customer	Delivery and product rating
order_value	Float (Engineered)	5.79 to 3773.42	Engineered	unit_price × quantity × (1 − discount)
discount_amount	Float (Engineered)	0.00 to 609.30	Engineered	Absolute monetary discount given
value_per_session_min	Float (Engineered)	0.31 to 599.62	Engineered	Order value per minute of browsing
late_delivery_flag	Binary (Engineered)	0 (< 7 days), 1 (≥ 7 days)	Engineered	Delivery took a week or longer
loyalty_score	Integer (Engineered)	0 to 25 points	Engineered	2×prior orders + 3×Prime + 2×(rating ≥ 4)
target	Binary Int	0 (One-time), 1 (Repeat)	Target	Customer made a repeat purchase

4. Dataset Description (Cont.)

4.2 Descriptive Summary Statistics of Continuous Variables

Table 2: Descriptive statistics (n = 2,000; missing values excluded)

Statistic	age	unit_price	quantity	discount %	ship days	order_value	session min	rating
Count	2000	2000	2000	2000	2000	2000	1978	1962
Mean	34.40	115.86	2.44	14.66	3.75	211.75	13.51	4.06
Std Dev	9.56	125.42	1.27	9.09	2.45	245.33	8.21	0.70
Median	34.00	69.72	2.00	14.40	3.00	139.29	12.00	4.10
Min	18.00	4.22	1.00	0.00	1.00	5.79	1.00	1.10
Q1 (25%)	27.00	39.34	1.00	8.00	2.00	71.56	7.50	3.60
Q3 (75%)	41.00	137.85	3.00	20.80	5.00	257.68	17.77	4.60
Max	65.00	1093.11	8.00	46.70	16.00	3773.42	57.60	5.00
Skewness	+0.26	+2.45	+0.92	+0.30	+1.15	+4.16	+1.17	-0.61
Kurtosis	-0.28	+7.85	+0.92	-0.40	+1.68	+33.16	+1.92	+0.06

Statistical Distribution Insights

1. Customer age ranges from 18 to 65 years (mean 34.40, median 34) and is nearly symmetric (skewness +0.26), describing a young-adult shopper base.

2. Price and order value are strongly right-skewed. Unit price has skewness +2.45 and order value +4.16 with kurtosis +33.16: the median order is 139.29 against a mean of 211.75, because a small number of high-priced Electronics baskets (maximum 3773.42) pull the mean upward. Median-based summaries and log scales are therefore more representative than the mean.

3. Delivery and engagement. Shipping time has median 3 days with a long tail to 16 days (skewness +1.15). Session length is also right-skewed (+1.17). Customer ratings are left-skewed (-0.61) with mean 4.06, so most customers are satisfied and low ratings form the minority tail.

5. EDA and Preprocessing

5.1 Preprocessing Pipeline Execution

1. Whitespace & Missing Value Treatment: Missing values occur in two attributes: customer_rating (38 values, 1.90%) and session_minutes (22 values, 1.10%). To avoid leakage, the data were split first and imputation used the training-partition median only (customer_rating = 4.1, session_minutes = 11.9). No orders were discarded.

2. Duplicate Record Audit: An audit across all columns except the order identifier found 0 duplicate rows.

3. Outlier Detection via Interquartile Range (IQR):

- **unit_price:** IQR = 98.51, lower bound = -108.42, upper bound = 285.61. Detected outliers = 192 (9.60%), maximum 1093.11.
- **order_value:** IQR = 186.12, lower bound = -207.63, upper bound = 536.87. Detected outliers = 145 (7.25%), maximum 3773.42.
- **shipping_days:** IQR = 3.0, lower bound = -2.5, upper bound = 9.5. Detected outliers = 58 (2.90%), maximum 16.00.
- **session_minutes:** IQR = 10.27, lower bound = -7.91, upper bound = 33.19. Detected outliers = 52 (2.60%), maximum 57.60.
- **discount_pct:** IQR = 12.8, lower bound = -11.2, upper bound = 40.0. Detected outliers = 5 (0.25%), maximum 46.70.

Business assessment: These outliers are genuine high-value purchases (premium electronics, multi-unit baskets), long cross-region deliveries and long browsing sessions rather than data-entry errors. They were retained, and standardization together with a log-scaled view in the figures keeps them from dominating the analysis.

4. Categorical Encoding: Binary attributes (is_prime, late_delivery_flag) were kept as {0, 1}. Nominal variables (gender, region, category, payment_method, device) were one-hot encoded with drop_first=True, expanding the feature space to **31 numeric input dimensions**.

5. Feature Standardization: The 13 continuous and count variables were standardized with StandardScaler (mean 0, standard deviation 1), fitted on the training partition and applied unchanged to the validation and test partitions.

6. Stratified Splitting: Orders were partitioned as shown in Table 3. Stratification keeps the repeat-purchase share near 63.8% in every subset.

Table 3: Stratified train / validation / test partitions

Partition	Share	Rows	One-time (0)	Repeat (1)
Training	70%	1400	506 (36.1%)	894 (63.9%)
Validation	15%	300	109 (36.3%)	191 (63.7%)
Test	15%	300	108 (36.0%)	192 (64.0%)

6. Data Visualization (Ben Fry's 7 Stages + 6 Exploratory Figures)

6.1 Ben Fry's 7 Stages of Data Visualization Framework

Table 4: Mapping of Fry's seven stages to this case study

Stage	Operational Implementation in this Case Study
1. Acquire	Generated a reproducible synthetic orders table (seed 42) into a Pandas DataFrame and saved it as ecommerce_sales_synthetic.csv.
2. Parse	Typed each field as continuous, discrete, binary or categorical, and defined the target (repeat_purchase → target).
3. Filter	Located missing values in customer_rating and session_minutes, verified duplicate independence, and flagged outliers with Tukey IQR fences.
4. Mine	Ran Welch t-tests, Mann-Whitney U tests, Chi-Square tests with Cramér's V, and Pearson correlations.
5. Represent	Chose encodings suited to each variable: bar and donut-style summaries, log-scaled histograms with KDE, violin and box plots, heatmaps, line and regression scatter plots.
6. Refine	Applied a consistent two-colour palette (#2B5F91 for one-time, #D94A38 for repeat), annotated values and exported figures at 200 DPI.
7. Interact	Interaction here is limited to the saved figures, CSV and results.json, which can be re-explored by re-running the script; an interactive dashboard is listed as future work.

6.2 Visualization 1: Target Distribution and Revenue by Category

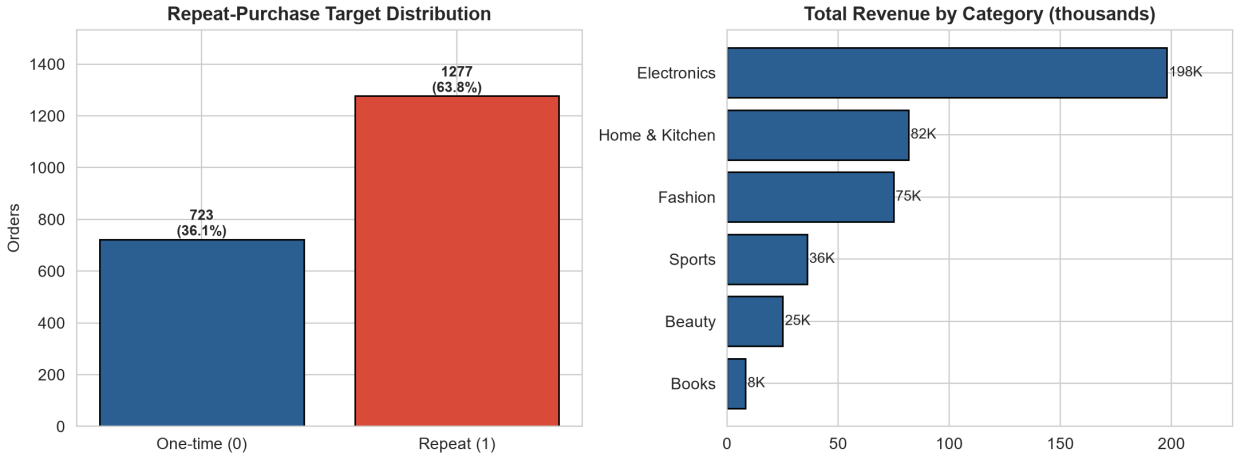


Figure 1: Repeat-purchase class distribution (left) and total revenue by product category (right)

What it shows: The count of one-time (723; 36.1%) and repeat (1277; 63.8%) orders, and total revenue by category.

Why it was created: To establish class priors before modelling and to see where revenue is concentrated.

Insights obtained: The class ratio is about 1.77:1 in favour of repeat purchase, so accuracy alone must be read against a majority-class baseline of 64.0%. Electronics produces 197,822 of 423,497 total revenue (46.7%) while Books contributes only 1.9%.

6. Data Visualization (Cont.)

6.3 Visualization 2: Continuous Probability Distributions (KDE & Histograms)

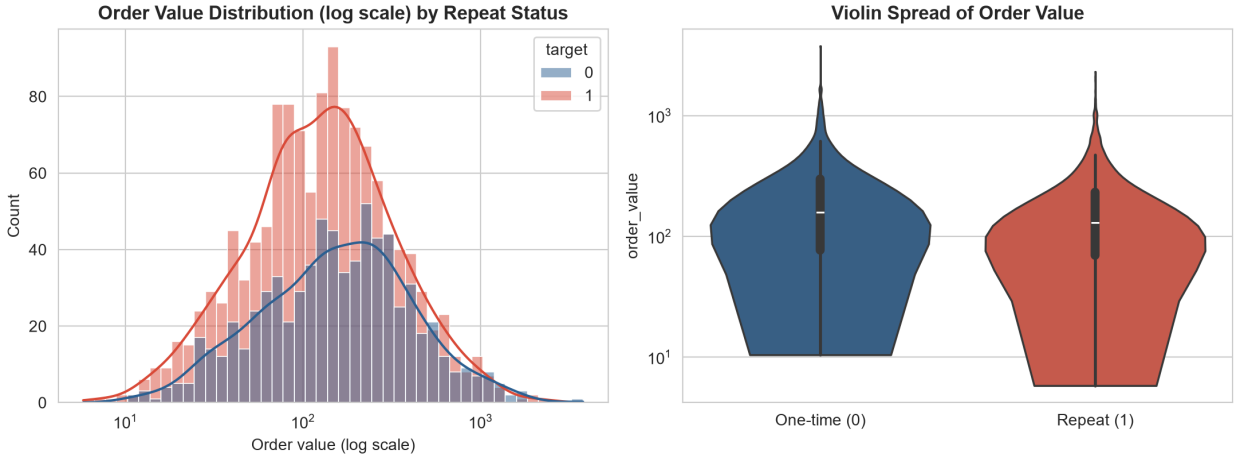


Figure 2: Log-scaled histogram with KDE and violin plot of order value by repeat status

What it shows: Empirical density, KDE curves and quartile spread of order value for one-time and repeat customers.

Why it was created: To check whether larger baskets are associated with returning, and to handle the heavy right tail with a log axis.

Insights obtained: Order value is roughly log-normal. Repeat customers have a lower mean order value (194.32) than one-time customers (242.52); Welch t = -3.95, p = 8.17e-05, and Mann-Whitney p = 3.26e-06. Expensive Electronics baskets are bought less often by returning customers.

6.4 Visualization 3: Bivariate Boxplots with Outlier Bounds

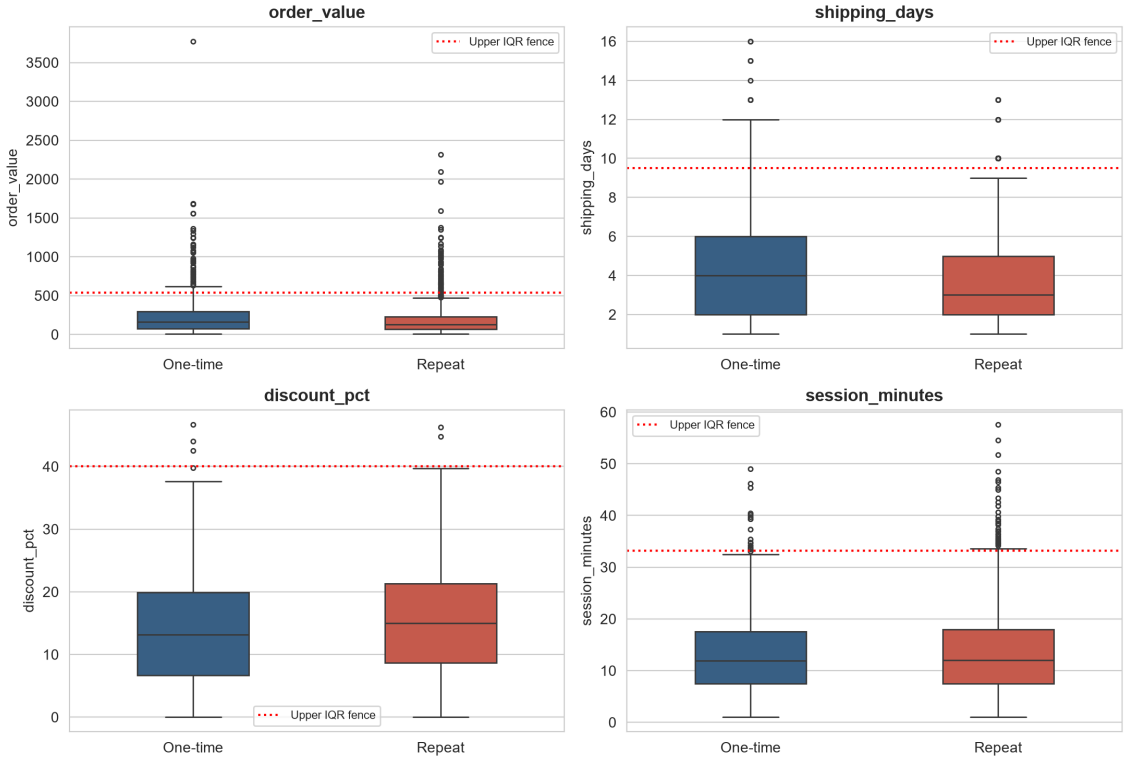


Figure 3: Multi-panel boxplots with Tukey IQR upper fences for order value, shipping days, discount and session length

What it shows: Medians, interquartile ranges and outlier fences for four numeric variables grouped by repeat status.

Insights obtained: Shipping time shows the clearest separation: repeat customers have a median of 3 days versus 4 for one-time customers (Welch t = -8.63, p = 1.79e-17). Discount differs modestly (15.14% vs 13.81%, p = 0.0018), whereas session length does not (p = 0.31).

6. Data Visualization (Cont.)

6.5 Visualization 4: Customer Segments vs Repeat Rate (Chi-Square Tests)

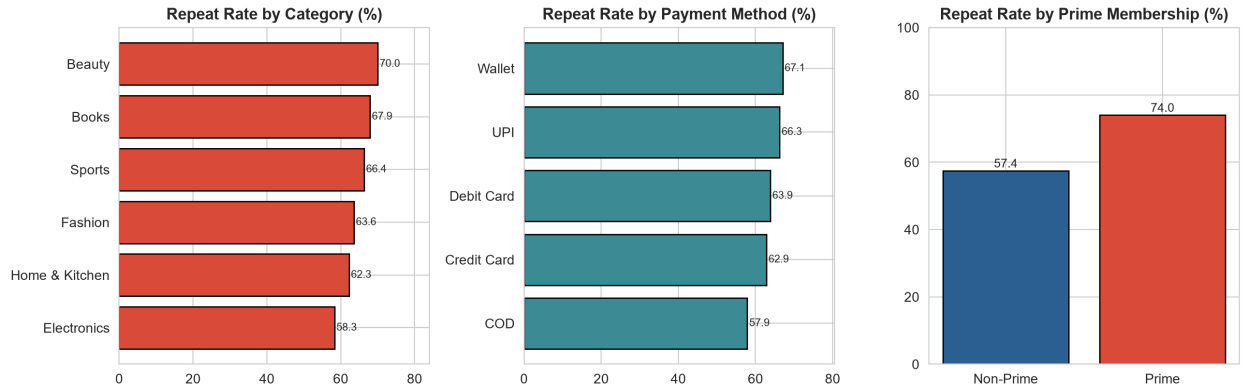


Figure 4: Repeat-purchase rate by product category, payment method and Prime membership

What it shows: The percentage of orders followed by a repeat purchase within each category, payment method and Prime group.

Why it was created: To find which categorical segments carry the strongest link to retention.

Insights obtained: Prime members repeat at 74.0% versus 57.4% for non-members ($\chi^2 = 55.54$, $p < 0.001$, Cramér's V = 0.167). Category is also significant ($\chi^2 = 12.76$, dof = 5, $p = 0.0257$, V = 0.08), with Beauty (70.0%) and Books (67.9%) highest and Electronics (58.3%) lowest. Payment method ($p = 0.142$), region ($p = 0.454$) and device ($p = 0.742$) are not statistically significant at the 5% level, although cash-on-delivery has the lowest repeat rate (57.9%).

6.6 Visualization 5: Multivariate Pearson Correlation Matrix

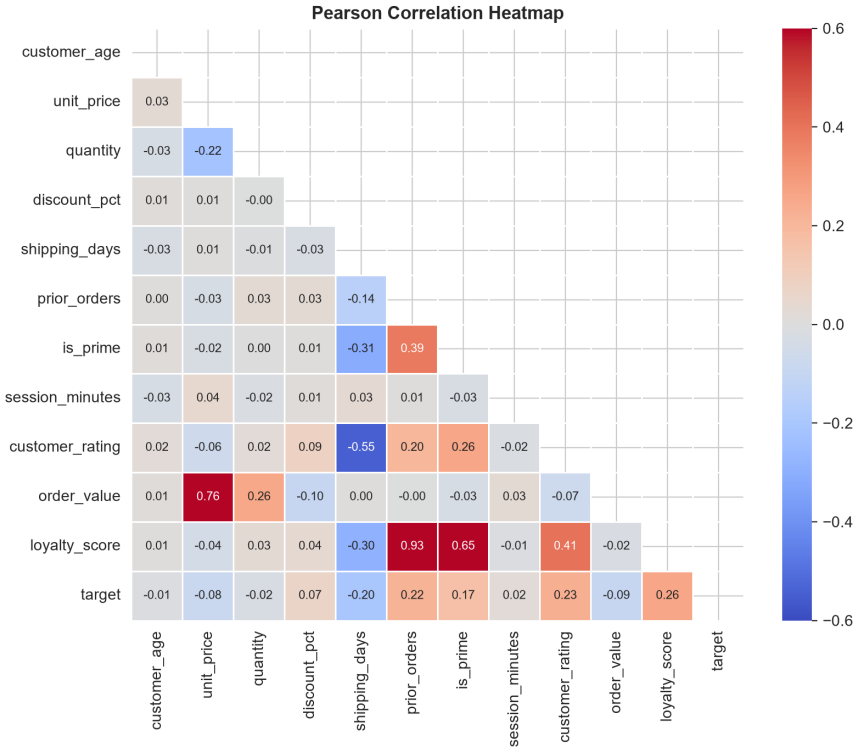


Figure 5: Lower-triangular Pearson correlation heatmap of order, customer and delivery variables with the target

Insights obtained: Linear associations with repeat purchase are modest. The strongest are customer_rating (+0.23), prior_orders (+0.22), is_prime (+0.17) and shipping_days (-0.20). Order value is weakly negative (-0.09). Correlation among predictors is low to moderate, so no variable was dropped.

6. Data Visualization (Cont.)

6.7 Visualization 6: Seasonality and Delivery Delay vs Rating

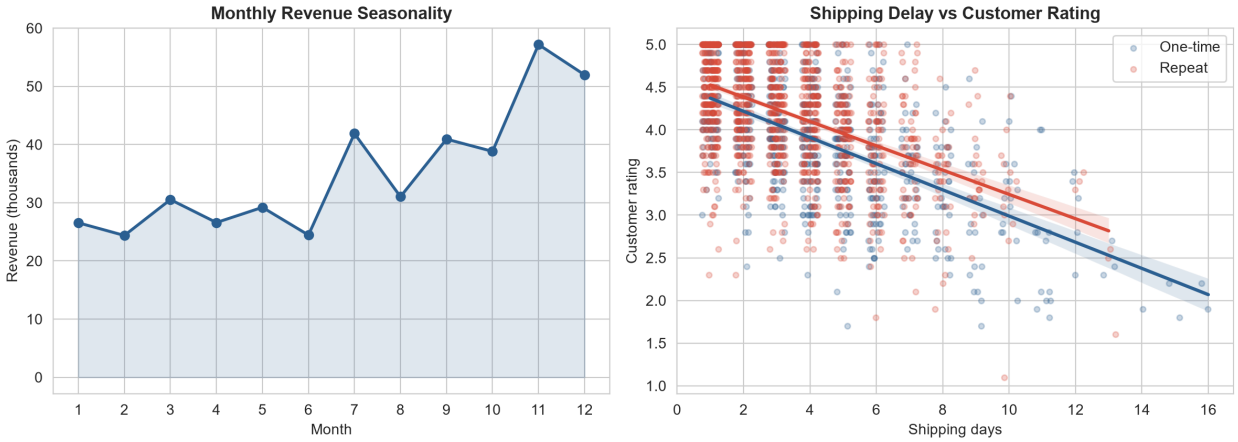


Figure 6: Monthly revenue seasonality (left) and shipping delay vs customer rating with regression lines (right)

What it shows: Revenue summed by calendar month, and the relationship between delivery time and customer rating for each customer group.

Why it was created: To quantify seasonal demand and to show how delivery performance feeds customer satisfaction.

Insights obtained: Revenue climbs sharply in the festive months: November (57,185) and December (51,976) together contribute 25.8% of annual revenue, against 16.7% expected if revenue were spread evenly across months. Shipping days and rating are moderately negatively correlated (Pearson $r = -0.544$, $p < 0.001$): every extra day of delivery is associated with lower satisfaction, which in turn relates to lower repeat probability.

Regional view: South (106,096) and North (91,700) lead revenue, while Central (60,869) trails and also has the longest delivery times by construction of the generator.

7. Deep Learning Model Building & Hyperparameter Tuning

7.1 Neural Network Formulation & Architecture

The classifier is a feed-forward multi-layer perceptron implemented with scikit-learn's MLPClassifier (TensorFlow was not available in the execution environment). It maps a standardized input vector x in R^{31} to a probability of repeat purchase:

$$\hat{y} = \sigma(W_3 \cdot \mathrm{ReLU}(W_2 \cdot \mathrm{ReLU}(W_1 x + b_1) + b_2) + b_3)$$

where $\sigma(z) = 1 / (1 + e^{-z})$. Training minimizes binary cross-entropy with an L2 weight penalty (parameter alpha), optimized by Adam with mini-batches of 32. scikit-learn's MLP has no dropout layer, so L2 regularization and early stopping provide the regularization.

Table 5: MLP layer specification for the selected configuration (Config C)

Layer	Input Dims	Output Dims	Activation / Regularization	Operational Role
Input Layer	31 features	64 units	Affine linear + ReLU	Projects scaled numeric and dummy features into a latent space
Hidden Layer 2	64 units	32 units	Dense + ReLU	Condenses interactions into a compact representation
Output Layer	32 units	1 unit	Sigmoid, log-loss	Probability of repeat purchase P(y=1 | x)
Regularization	—	—	L2 alpha = 0.001; early stopping (patience 15)	Limits overfitting on 1,400 training rows

7.2 Hyperparameter Tuning Experiment

Three candidate architectures were trained on the training partition and compared on the validation partition. Each run used manual epoch-by-epoch training with early stopping (patience = 15 epochs) and restoration of the weights from the epoch with the lowest validation loss.

Table 6: Hyperparameter grid and validation results

Configuration	Hidden Layers	Learning Rate	L2 alpha	Best Epoch	Best Val Loss	Val ROC-AUC	Params
Config A (Shallow Fast)	(32)	0.005	0.001	4	0.6196	0.6738	1,057
Config B (Tuned Deep MLP)	(32, 16)	0.001	0.01	6	0.6186	0.6650	1,569
Config C (Wide Deep MLP) ★	(64, 32)	0.0005	0.001	9	0.6123	0.6855	4,161

Selection Rationale: Config C (Wide Deep MLP) was selected because it achieved the lowest validation cross-entropy (0.6123) and the highest validation ROC-AUC (0.6855). The three configurations differ only slightly (validation loss between 0.6123 and 0.6196), so the choice is not strongly decisive; with 300 validation rows, such small gaps lie within sampling noise. The selected model is also the largest (4,161 trainable parameters).

7. Deep Learning Model Building (Cont.)

7.3 Permutation Feature Importance Analysis

Model-agnostic permutation importance was computed on the held-out test partition (N = 300). Each feature is shuffled 10 times and the mean drop in test ROC-AUC is recorded:

Table 7: Top-10 permutation feature importances

Rank	Feature Name	Test ROC-AUC Drop (Importance)	Standard Deviation
#1	loyalty_score	0.03819	± 0.01110
#2	prior_orders	0.02109	± 0.01084
#3	shipping_days	0.01461	± 0.01069
#4	customer_rating	0.01150	± 0.00811
#5	unit_price	0.00537	± 0.00619
#6	is_prime	0.00521	± 0.00301
#7	category_Fashion	0.00466	± 0.00303
#8	discount_amount	0.00379	± 0.00463
#9	value_per_session_min	0.00339	± 0.00271
#10	device_Mobile	0.00286	± 0.00159

Interpretability & Synthesis

1. Loyalty drives the prediction. The engineered loyalty_score (built from prior orders, Prime status and high rating) is the top feature (AUC drop 0.0382), followed by prior_orders (0.0211). Customers with an order history are the most likely to come back.

2. Delivery experience matters. shipping_days ranks third and customer_rating fourth, consistent with the statistical tests in Section 6: slow delivery and poor ratings reduce repeat purchase.

3. Caution on interpretation. loyalty_score overlaps with prior_orders, is_prime and customer_rating, so permuting one of them can be partly compensated by the others; importance is shared across correlated features. All importances are small (the largest drop is under 0.04 AUC) and several have standard deviations of similar size, so only the top few rankings should be treated as reliable.

8. Result Visualization & Interpretation (Step 5 Evaluation)

8.1 Figure 7: Training & Validation Loss vs Epoch Trajectory

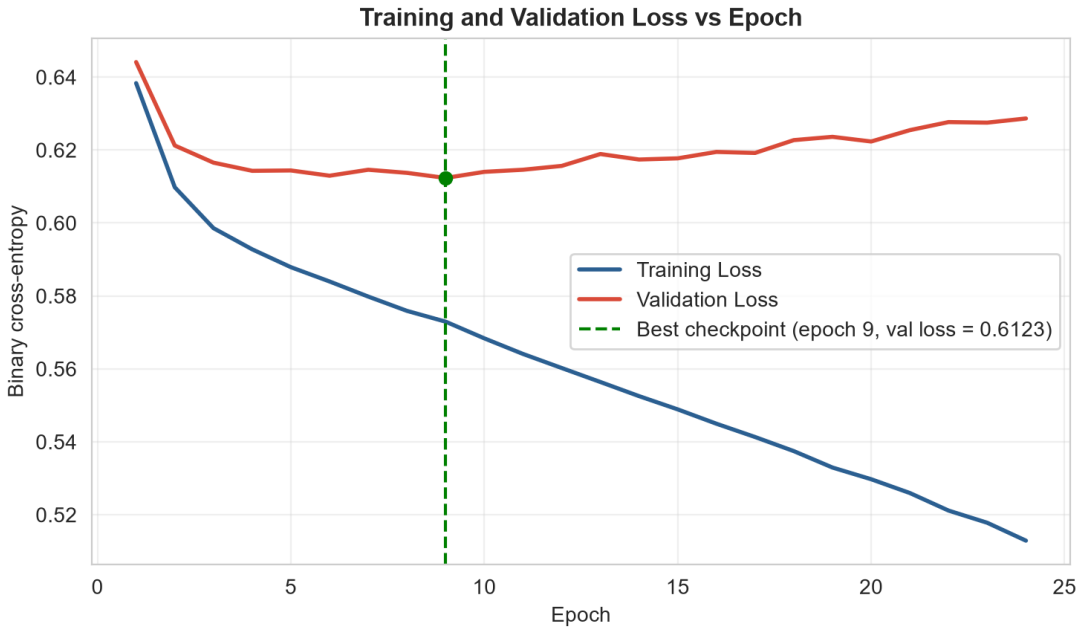


Figure 7: Training and validation binary cross-entropy loss vs epoch with optimal checkpoint

Chart Description: Training and validation log-loss for the selected configuration, with the lowest-validation-loss epoch marked.

Performance Interpretation: Training loss falls steadily from 0.638 to 0.513 over 24 epochs. Validation loss reaches its minimum of 0.6123 at epoch 9 and then rises, so early stopping triggered and the epoch-9 weights were restored. The widening gap shows the network begins to memorize the 1,400 training orders once the generalizable signal has been learned, which is expected for a dataset of this size and signal strength.

8. Result Visualization & Interpretation (Cont.)

8.2 Figure 8: Training & Validation Accuracy vs Epoch Trajectory

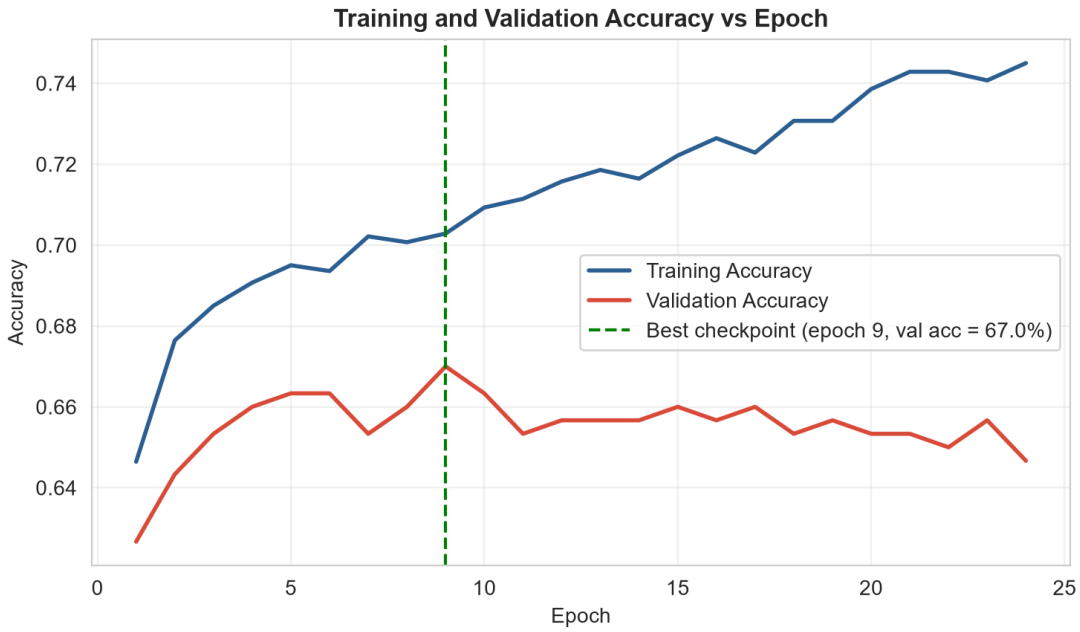


Figure 8: Training and validation accuracy across learning epochs

Performance Interpretation: Training accuracy rises to 74.5% while validation accuracy stays near 67%. The validation figure is only a few points above the majority-class rate of 64.0%, so most of the model's apparent skill lies in ranking customers (see ROC-AUC) rather than in hard classification at a 0.5 cutoff.

8.3 Figure 9: Out-of-Sample Test Confusion Matrix

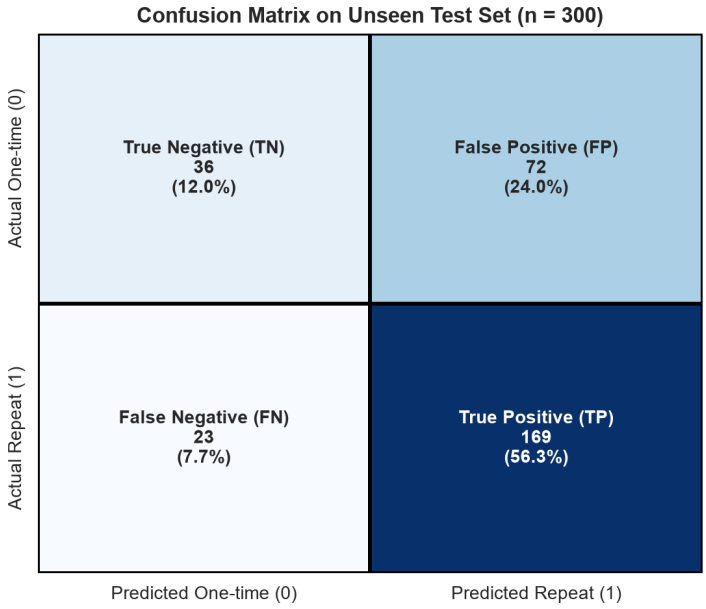


Figure 9: Confusion matrix on the held-out test set (n = 300)

Performance Interpretation: At the default 0.5 threshold the model correctly identifies **169 repeat buyers** (recall 88.0%) and 36 one-time buyers (specificity 33.3%). It misses 23 repeat customers (false negatives) and wrongly flags 72 one-time customers as repeat (false positives). Because repeat customers are the majority class, the model leans toward predicting repeat purchase.

8. Result Visualization & Interpretation (Cont.)

8.4 Figure 10: Receiver Operating Characteristic (ROC) & AUC Comparison

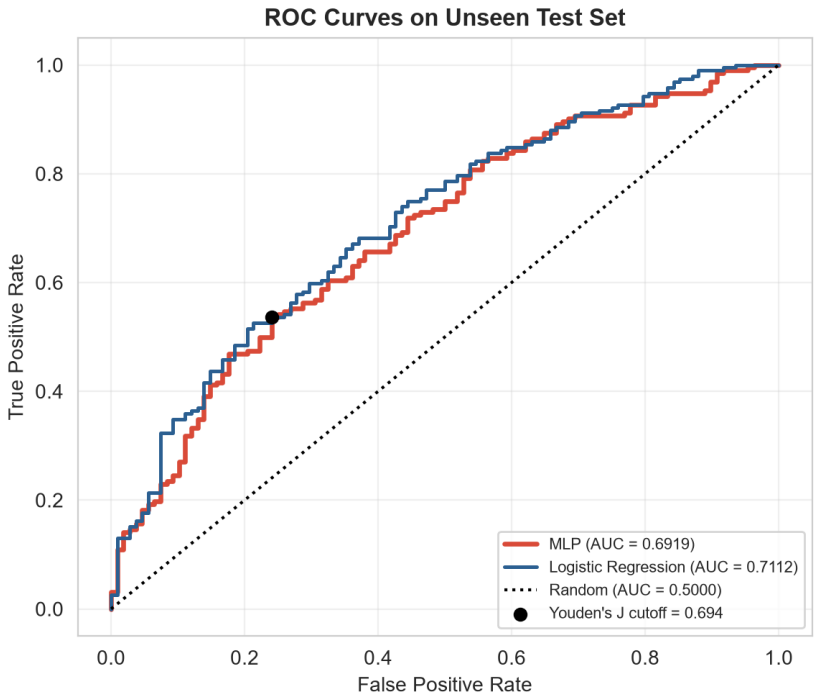


Figure 10: ROC curves for the MLP and logistic regression with Youden's J operating point

Performance Interpretation: The MLP achieves a test ROC-AUC of **0.6919**, clearly above random guessing (0.5000), and the logistic regression baseline reaches 0.7112. Youden's J picks a cutoff of 0.694, giving 53.6% sensitivity at a 24.1% false-positive rate. This operating point trades recall for far fewer false alarms than the 0.5 cutoff. The two ROC curves overlap closely, indicating the signal is mostly linear.

8.5 Comprehensive Out-of-Sample Benchmark Comparison Table

Table 8: Test-set benchmark – MLP vs Logistic Regression baseline (n = 300, 0.5 threshold)

Evaluation Metric	MLP	Logistic Regression	Delta (MLP − LR)
Overall Accuracy	68.33%	68.67%	-0.33 pts
Precision (Positive)	70.12%	70.25%	-0.12 pts
Recall / Sensitivity	88.02%	88.54%	-0.52 pts
Specificity	33.33%	33.33%	+0.00 pts
F1-Score	0.7806	0.7834	-0.0028
ROC-AUC	0.6919	0.7112	-0.0193
Average Precision (PR-AUC)	0.7947	0.8078	-0.0131
Majority-class accuracy	64.00%	64.00%	—

Comparative Synthesis

The neural network does **not** outperform the linear baseline on this dataset: logistic regression is marginally ahead on ROC-AUC (0.7112 vs 0.6919), F1 and accuracy, while the two tie on specificity. With 300 test rows, differences of this size are within sampling noise, so the fair reading is that both models perform similarly. Both beat the majority-class baseline in AUC, but accuracy gains are small (+4.3 points over 64.0%). This is consistent with how the data were generated: repeat purchase depends mostly on additive effects (loyalty, shipping speed, Prime membership) plus substantial random noise, a setting where extra non-linear capacity adds little.

9. Conclusion & Future Scope

9.1 Key Empirical Findings & Patterns

1. Loyalty and Prime membership matter most. Prime customers repeat at 74.0% versus 57.4% ($\chi^2 = 55.54$, $p < 0.001$), and repeat customers have placed more prior orders (mean 3.06 vs 2.22; $p < 0.001$).

2. Delivery speed protects retention. Repeat customers waited 3.38 days on average versus 4.40, and slower delivery lowers ratings ($r = -0.544$).

3. Revenue is concentrated. Electronics accounts for 46.7% of revenue but has the lowest repeat rate (58.3%); Beauty and Books retain best. November and December together bring 25.8% of revenue.

9.2 Model Performance Summary

The tuned MLP (Config C) reached 68.33% test accuracy, 88.02% recall, 33.33% specificity and ROC-AUC 0.6919 on unseen data, comparable to logistic regression (0.7112). Imputation and scaling were fitted on the training partition only, and the test set was used once, for final evaluation.

9.3 Technical Challenges Faced

• Heavy right-skew in order value and unit price required log-scaled plots and median-based summaries.

• Moderate class imbalance means accuracy must be compared with the majority-class baseline.

• A small dataset (2,000 orders) with noisy labels limits what any model can learn; the MLP over-fit after about nine epochs.

• TensorFlow/Keras was unavailable, so scikit-learn's MLP was used and L2 regularization replaced dropout.

9.4 Future Improvements & Technical Extensions

• **Real-data validation:** repeat the pipeline on a public retail dataset (for example Olist or Superstore) to confirm the patterns.

• **Gradient boosting and calibration:** compare against tree ensembles and calibrate probabilities before using them for targeting.

• **Cost-sensitive thresholding:** choose the cutoff from the expected value of a retention offer instead of defaulting to 0.5.

• **Interactive dashboard:** publish the figures as a filterable dashboard to complete Fry's Interact stage.

10. Literature Review & References

The methodology, statistical framework and visualization workflow follow the established literature below:

[1] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA: O'Reilly Media, 2008.

[2] J. W. Tukey, Exploratory Data Analysis. Reading, MA: Addison-Wesley, 1977.

[3] B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," Biometrika, vol. 34, no. 1–2, pp. 28–35, 1947.

[4] H. Cramér, Mathematical Methods of Statistics. Princeton, NJ: Princeton University Press, 1946.

[5] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conference, 2010, pp. 56–61.

[8] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

11. Appendix: Core Reproducible Code Architecture

The full pipeline, covering data generation, preprocessing, statistical tests, all figures, MLP tuning, evaluation and permutation importance, is one script (analysis.py, run with Python 3.12, scikit-learn, pandas, SciPy, seaborn and matplotlib). It writes the dataset CSV, ten figures and results.json. Long lines are wrapped for print.

```
import json, copy, warnings
import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, log_loss, average_precision_score)
warnings.filterwarnings("ignore")
SEED=42; rng=np.random.default_rng(SEED); np.random.seed(SEED)
PAL={0:"#2B5F91",1:"#D94A38"}; sns.set_style("whitegrid")
R={}
# ---------- Stage 1-2: Acquire (synthetic) ----------
N=2000
cats=["Electronics","Fashion","Home & Kitchen","Beauty","Books","Sports"]
cat_p=[.22,.26,.18,.14,.10,.10]
base_price={"Electronics":280,"Fashion":55,"Home & Kitchen":90,"Beauty":35,"Books":18,"Sports":70}
regions=["North","South","East","West","Central"]
pays=["Credit Card","UPI","Debit Card","COD","Wallet"]
devs=["Mobile","Desktop","Tablet"]
df=pd.DataFrame({
 "order_month":rng.choice(np.arange(1,13),N,p=(lambda
     w:w/w.sum())(np.array([6,6,7,7,7,7,8,8,8,10,13,13],dtype=float))),
 "customer_age":np.clip(rng.normal(34,10,N),18,70).round().astype(int),
 "gender":rng.choice(["F","M"],N),
 "region":rng.choice(regions,N,p=[.2,.25,.2,.2,.15]),
 "category":rng.choice(cats,N,p=cat_p),
 "payment_method":rng.choice(pays,N,p=[.28,.34,.16,.14,.08]),
 "device":rng.choice(devs,N,p=[.62,.30,.08]),
})
df["unit_price"]=[round(float(rng.lognormal(np.log(base_price[c]),0.45)),2) for c in df.category]
df["quantity"]=np.clip(rng.poisson(1.6,N)+1-(df.category=="Electronics")*1,1,8)
df["discount_pct"]=np.clip(rng.normal(12,9,N)+(df.order_month>=11)*8,0,60).round(1)
df["shipping_days"]=np.clip(rng.gamma(3,1.4,N)+(df.region=="Central")*1.5,1,18).round().astype(int)
df["is_prime"]=rng.binomial(1,.38,N)
df.loc[df.is_prime==1,"shipping_days"]=np.clip(df.loc[df.is_prime==1,"shipping_days"]-2,1,18)
df["prior_orders"]=rng.poisson(2.2+1.5*df.is_prime,N)
df["session_minutes"]=np.clip(rng.gamma(2.5,5,N)+(df.device=="Desktop")*3,1,90).round(1)
cat_eff=df.category.map({"Electronics":0,"Fashion":.1,"Home & Kitchen":.0,"Beauty":.25,"Books":.35,"Sports":.05})
lp=(-0.2+0.22*df.prior_orders+0.5*df.is_prime-0.17*(df.shipping_days-4)+0.025*df.discount_pct
    +cat_eff-0.0012*(df.unit_price*df.quantity).clip(upper=600)+0.012*(df.session_minutes-15)
    -0.3*(df.payment_method=="COD")+rng.normal(0,0.9,N))
df["customer_rating"]=np.clip((4.1-0.12*(df.shipping_days-4)+0.3*(lp-
    lp.mean())/lp.std()+rng.normal(0,.6,N)),1,5).round(1)
df["repeat_purchase"]=(rng.random(N)<1/(1+np.exp(-lp))).astype(int)
# missing values (real-world style)
df.loc[rng.choice(N,38,replace=False),"customer_rating"]=np.nan
df.loc[rng.choice(N,22,replace=False),"session_minutes"]=np.nan
# engineered
df["order_value"]=(df.unit_price*df.quantity*(1-df.discount_pct/100)).round(2)
df["discount_amount"]=(df.unit_price*df.quantity*df.discount_pct/100).round(2)
df["value_per_session_min"]=(df.order_value/df.session_minutes).round(2)
df["late_delivery_flag"]=(df.shipping_days>=7).astype(int)
df["loyalty_score"]=(df.prior_orders*2+df.is_prime*3+(df.customer_rating.fillna(df.customer_rating.median())>=4).astyp
    e(int)*2)
df["target"]=df.repeat_purchase
df.insert(0,"order_id",["ORD%05d"%i for i in range(1,N+1)])
df.to_csv("data/ecommerce_sales_synthetic.csv",index=False)
R["shape"]=df.shape; R["n_features"]=df.shape[1]-2
R["dups"]=int(df.duplicated(subset=[c for c in df.columns if c!="order_id"]).sum())
R["missing"]={k:int(v) for k,v in df.isnull().sum().items() if v>0}
R["class_counts"]=df.target.value_counts().sort_index().to_dict()
cont=["customer_age","unit_price","quantity","discount_pct","shipping_days","order_value","session_minutes","customer_
    rating"]
desc=df[cont].describe().T
desc["skew"]=df[cont].skew(); desc["kurt"]=df[cont].kurt()
R["desc"]=desc.round(2).reset_index().to_dict("records")
# outliers
out={}
for c in ["unit_price","order_value","shipping_days","session_minutes","discount_pct"]:
    q1,q3=df[c].quantile([.25,.75]); i=q3-q1; lo,hi=q1-1.5*i,q3+1.5*i
out[c]=dict(iqr=round(i,2),lo=round(lo,2),hi=round(hi,2),n=int(((df[c]<lo)|(df[c]>hi)).sum()),mx=float(df[c].max()))
R["outliers"]=out
# ---------- Split ----------
feat=df.drop(columns=["order_id","repeat_purchase","target"])
y=df.target.values
Xtr,Xtmp,ytr,ytmp=train_test_split(feat,y,test_size=.30,stratify=y,random_state=SEED)
Xva,Xte,yva,yte=train_test_split(Xtmp,ytmp,test_size=.50,stratify=ytmp,random_state=SEED)
R["split"]={k:[len(a),int((b==0).sum()),int((b==1).sum())] for k,a,b in
    [("train",Xtr,ytr),("val",Xva,yva),("test",Xte,yte)]}
for c in ["customer_rating","session_minutes"]:
    med=Xtr[c].median()
    for p in (Xtr,Xva,Xte): p[c]=p[c].fillna(med)
R["impute"]={c:round(float(Xtr[c].median()),2) for c in ["customer_rating","session_minutes"]}
```

11. Appendix: Core Code Architecture (Cont.)

```
for p in (Xtr,Xva,Xte):
    p["value_per_session_min"]=(p.order_value/p.session_minutes).round(2)
    p["loyalty_score"]=p.prior_orders*2+p.is_prime*3+(p.customer_rating>=4).astype(int)*2
catc=["gender","region","category","payment_method","device"]
Etr=pd.get_dummies(Xtr,columns=catc,drop_first=True)
Eva=pd.get_dummies(Xva,columns=catc,drop_first=True).reindex(columns=Etr.columns,fill_value=0)
Ete=pd.get_dummies(Xte,columns=catc,drop_first=True).reindex(columns=Etr.columns,fill_value=0)
numc=["order_month","customer_age","unit_price","quantity","discount_pct","shipping_days","prior_orders","session_minu
    tes","customer_rating","order_value","discount_amount","value_per_session_min","loyalty_score"]
sc=StandardScaler(); Etr[numc]=sc.fit_transform(Etr[numc]); Eva[numc]=sc.transform(Eva[numc]);
    Ete[numc]=sc.transform(Ete[numc])
Etr,Eva,Ete=[e.astype(float) for e in (Etr,Eva,Ete)]
R["input_dim"]=Etr.shape[1]
# ---------- Stats tests ----------
t={}
for c in ["order_value","shipping_days","discount_pct","prior_orders","session_minutes"]:
    a=df[df.target==1][c].dropna(); b=df[df.target==0][c].dropna()
    tt=stats.ttest_ind(a,b,equal_var=False); mw=stats.mannwhitneyu(a,b)
    t[c]=dict(m1=round(a.mean(),2),m0=round(b.mean(),2),med1=round(a.median(),2),med0=round(b.median(),2),t=round(tt.s
        tatistic,2),p=float(tt.pvalue),mwp=float(mw.pvalue))
R["ttests"]=t
chi={}
for c in ["category","region","payment_method","device","is_prime"]:
    ct=pd.crosstab(df[c],df.target); chi2,p,dof,_=stats.chi2_contingency(ct)
    v=np.sqrt(chi2/(ct.values.sum()*(min(ct.shape)-1)))
    chi[c]=dict(chi2=round(chi2,2),p=float(p),dof=int(dof),v=round(v,3))
R["chi"]=chi
R["rate_by_cat"]=(df.groupby("category").target.mean()*100).round(1).to_dict()
R["rate_by_pay"]=(df.groupby("payment_method").target.mean()*100).round(1).to_dict()
R["rate_prime"]=(df.groupby("is_prime").target.mean()*100).round(1).to_dict()
R["rev_by_cat"]=df.groupby("category").order_value.sum().round(0).sort_values(ascending=False).to_dict()
R["rev_total"]=round(float(df.order_value.sum()),0); R["aov"]=round(float(df.order_value.mean()),2)
R["rev_by_month"]=df.groupby("order_month").order_value.sum().round(0).to_dict()
R["rev_by_region"]=df.groupby("region").order_value.sum().round(0).to_dict()
corr=df[cont+["prior_orders","is_prime","target"]].corr()
R["corr_target"]=corr.target.drop("target").round(2).sort_values().to_dict()
r_late=stats.pearsonr(df.shipping_days,df.customer_rating.fillna(df.customer_rating.median()))
R["ship_rating_r"]=[round(r_late[0],3),float(r_late[1])]
# ---------- Figures ----------
def sv(n): plt.savefig(f"fig/{n}.png",dpi=200,bbox_inches="tight"); plt.close()
# F1 class dist + revenue by category
fig,ax=plt.subplots(1,2,figsize=(11,4.2))
c=df.target.value_counts().sort_index()
b=ax[0].bar(["One-time (0)","Repeat (1)"],c.values,color=[PAL[0],PAL[1]],edgecolor="k")
for r,v in zip(b,c.values):
    ax[0].text(r.get_x()+r.get_width()/2,v+15,f"{v}\n({v/N*100:.1f}%)",ha="center",fontsize=9,fontweight="bold")
ax[0].set_ylim(0,c.max()*1.2); ax[0].set_ylabel("Orders"); ax[0].set_title("Repeat-Purchase Target
    Distribution",fontweight="bold")
rv=df.groupby("category").order_value.sum().sort_values()
ax[1].barh(rv.index,rv.values/1000,color="#2B5F91",edgecolor="k")
for i,v in enumerate(rv.values/1000): ax[1].text(v+1,i,f"{v:,.0f}K",va="center",fontsize=9)
ax[1].set_title("Total Revenue by Category (thousands)",fontweight="bold"); ax[1].set_xlim(0,rv.max()/1000*1.15)
plt.tight_layout(); sv("f1")
# F2 order value distribution
fig,ax=plt.subplots(1,2,figsize=(11,4.2))
sns.histplot(data=df,x="order_value",hue="target",palette=PAL,kde=True,bins=45,ax=ax[0],log_scale=True)
ax[0].set_title("Order Value Distribution (log scale) by Repeat Status",fontweight="bold"); ax[0].set_xlabel("Order
    value (log scale)")
sns.violinplot(data=df,x="target",y="order_value",hue="target",palette=PAL,ax=ax[1],cut=0,legend=False)
ax[1].set_yscale("log"); ax[1].set_xticklabels(["One-time (0)","Repeat (1)"]); ax[1].set_title("Violin Spread of Order
    Value",fontweight="bold"); ax[1].set_xlabel("")
plt.tight_layout(); sv("f2")
# F3 boxplots
fig,axes=plt.subplots(2,2,figsize=(11,7.5))
for ax,cc in zip(axes.flat,["order_value","shipping_days","discount_pct","session_minutes"]):
    sns.boxplot(data=df,x="target",y=cc,hue="target",palette=PAL,ax=ax,width=.45,fliersize=3,legend=False)
    q1,q3=df[cc].quantile([.25,.75]); ax.axhline(q3+1.5*(q3-q1),color="red",ls=":",label="Upper IQR fence")
    ax.set_title(cc,fontweight="bold"); ax.set_xticklabels(["One-time","Repeat"]); ax.set_xlabel("");
        ax.legend(fontsize=8)
plt.tight_layout(); sv("f3")
# F4 category/prime/payment rates
fig,ax=plt.subplots(1,3,figsize=(13,4.2))
rc=df.groupby("category").target.mean().sort_values()*100
ax[0].barh(rc.index,rc.values,color="#D94A38",edgecolor="k"); ax[0].set_title("Repeat Rate by Category
    (%)",fontweight="bold")
for i,v in enumerate(rc.values): ax[0].text(v+.5,i,f"{v:.1f}",va="center",fontsize=8)
ax[0].set_xlim(0,rc.max()*1.2)
rp=df.groupby("payment_method").target.mean().sort_values()*100
ax[1].barh(rp.index,rp.values,color="#3B8A94",edgecolor="k"); ax[1].set_title("Repeat Rate by Payment Method
    (%)",fontweight="bold")
for i,v in enumerate(rp.values): ax[1].text(v+.5,i,f"{v:.1f}",va="center",fontsize=8)
ax[1].set_xlim(0,rp.max()*1.2)
rpr=df.groupby("is_prime").target.mean()*100
ax[2].bar(["Non-Prime","Prime"],rpr.values,color=["#2B5F91","#D94A38"],edgecolor="k"); ax[2].set_title("Repeat Rate by
    Prime Membership (%)",fontweight="bold")
for i,v in enumerate(rpr.values): ax[2].text(i,v+1,f"{v:.1f}",ha="center",fontsize=9)
ax[2].set_ylim(0,100)
plt.tight_layout(); sv("f4")
```

11. Appendix: Core Code Architecture (Cont.)

```
# F5 correlation
cc=["customer_age","unit_price","quantity","discount_pct","shipping_days","prior_orders","is_prime","session_minutes",
    "customer_rating","order_value","loyalty_score","target"]
cm=df[cc].corr(); mask=np.triu(np.ones_like(cm,dtype=bool))
plt.figure(figsize=(9,7));
    sns.heatmap(cm,mask=mask,annot=True,fmt=".2f",cmap="coolwarm",vmin=-.6,vmax=.6,linewidths=.5,annot_kws={"size":8})
plt.title("Pearson Correlation Heatmap",fontweight="bold"); sv("f5")
# F6 monthly revenue + shipping vs rating
fig,ax=plt.subplots(1,2,figsize=(12,4.4))
mr=df.groupby("order_month").order_value.sum()/1000
ax[0].plot(mr.index,mr.values,marker="o",color="#2B5F91",lw=2);
    ax[0].fill_between(mr.index,mr.values,alpha=.15,color="#2B5F91")
ax[0].set_xticks(range(1,13)); ax[0].set_xlabel("Month"); ax[0].set_ylabel("Revenue (thousands)");
    ax[0].set_title("Monthly Revenue Seasonality",fontweight="bold")
tmp=df.assign(r=df.customer_rating)
sns.regplot(data=tmp[tmp.target==0],x="shipping_days",y="r",x_jitter=.25,scatter_kws=dict(alpha=.25,s=14),color=PAL[0]
    ,ax=ax[1],label="One-time")
sns.regplot(data=tmp[tmp.target==1],x="shipping_days",y="r",x_jitter=.25,scatter_kws=dict(alpha=.25,s=14),color=PAL[1]
    ,ax=ax[1],label="Repeat")
ax[1].set_xlabel("Shipping days"); ax[1].set_ylabel("Customer rating"); ax[1].set_title("Shipping Delay vs Customer
    Rating",fontweight="bold"); ax[1].legend()
plt.tight_layout(); sv("f6")
# ---------- Models ----------
cfgs={"Config A (Shallow Fast)":dict(h=(32,),a=1e-3,lr=5e-3),
      "Config B (Tuned Deep MLP)":dict(h=(32,16),a=1e-2,lr=1e-3),
      "Config C (Wide Deep MLP)":dict(h=(64,32),a=1e-3,lr=5e-4)}
def train(cfg,epochs=150,patience=15):
    m=MLPClassifier(hidden_layer_sizes=cfg["h"],alpha=cfg["a"],learning_rate_init=cfg["lr"],batch_size=32,random_state
        =SEED,max_iter=1)
    cls=np.array([0,1]); hist=dict(tl=[],vl=[],ta=[],va=[]); best=(1e9,None,0); bad=0
    for ep in range(1,epochs+1):
        idx=np.random.RandomState(SEED+ep).permutation(len(Etr))
        m.partial_fit(Etr.values[idx],ytr[idx],classes=cls)
        ptr=m.predict_proba(Etr)[:,1]; pva=m.predict_proba(Eva)[:,1]
        hist["tl"].append(log_loss(ytr,ptr)); hist["vl"].append(log_loss(yva,pva))
        hist["ta"].append(accuracy_score(ytr,ptr>.5)); hist["va"].append(accuracy_score(yva,pva>.5))
        if hist["vl"][-1]<best[0]-1e-4: best=(hist["vl"][-1],copy.deepcopy(m),ep); bad=0
        else: bad+=1
        if bad>=patience: break
    return best,hist
res={}
for k,cfg in cfgs.items():
    best,h=train(cfg); mm=best[1]
    pva=mm.predict_proba(Eva)[:,1]
    nparams=sum(w.size for w in mm.coefs_)+sum(b.size for b in mm.intercepts_)
    res[k]=dict(loss=best[0],auc=roc_auc_score(yva,pva),ep=best[2],np=int(nparams),hist=h,model=mm,cfg=cfg)
bk=min(res,key=lambda k:res[k]["loss"]); R["tuning"]={k:dict(h=list(v["cfg"]["h"]),lr=v["cfg"]["lr"],alpha=v["cfg"]["a
    "],loss=round(v["loss"],4),auc=round(v["auc"],4),ep=v["ep"],np=v["np"]) for k,v in res.items()}
R["best_cfg"]=bk
mlp=res[bk]["model"]; H=res[bk]["hist"]; be=res[bk]["ep"]; R["best_epoch"]=be; R["epochs_run"]=len(H["tl"])
R["best_val_acc"]=round(H["va"][be-1],4); R["best_val_loss"]=round(H["vl"][be-1],4)
R["train_loss0"]=round(H["tl"][0],3); R["train_loss_end"]=round(H["tl"][-1],3);
    R["train_acc_end"]=round(H["ta"][-1],3)
# curves
ep=np.arange(1,len(H["tl"])+1)
plt.figure(figsize=(8.5,4.6)); plt.plot(ep,H["tl"],lw=2,color="#2B5F91",label="Training Loss");
    plt.plot(ep,H["vl"],lw=2,color="#D94A38",label="Validation Loss")
plt.axvline(be,color="green",ls="--",label=f"Best checkpoint (epoch {be}, val loss = {H['vl'][be-1]:.4f})");
    plt.scatter([be],[H["vl"][be-1]],color="green",zorder=5)
plt.xlabel("Epoch"); plt.ylabel("Binary cross-entropy"); plt.title("Training and Validation Loss vs
    Epoch",fontweight="bold"); plt.legend(); plt.grid(alpha=.3); sv("f7")
plt.figure(figsize=(8.5,4.6)); plt.plot(ep,H["ta"],lw=2,color="#2B5F91",label="Training Accuracy");
    plt.plot(ep,H["va"],lw=2,color="#D94A38",label="Validation Accuracy")
plt.axvline(be,color="green",ls="--",label=f"Best checkpoint (epoch {be}, val acc = {H['va'][be-1]*100:.1f}%)")
plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.title("Training and Validation Accuracy vs Epoch",fontweight="bold");
    plt.legend(); plt.grid(alpha=.3); sv("f8")
# test eval
pt=mlp.predict_proba(Ete)[:,1]
fpr,tpr,thr=roc_curve(yte,pt); J=tpr-fpr; jt=thr[J.argmax()]
def ev(p,th=.5):
    pr=(p>=th).astype(int); tn,fp,fn,tp=confusion_matrix(yte,pr).ravel()
    return dict(acc=accuracy_score(yte,pr),prec=precision_score(yte,pr),rec=recall_score(yte,pr),spec=tn/(tn+fp),f1=f1
        _score(yte,pr),auc=roc_auc_score(yte,p),ap=average_precision_score(yte,p),tn=int(tn),fp=int(fp),fn=int(fn),tp=
        int(tp))
R["mlp"]=ev(pt);
    R["mlp_youden"]=dict(thr=round(float(jt),3),tpr=round(float(tpr[J.argmax()]),3),fpr=round(float(fpr[J.argmax()]),3
    ))
lr=LogisticRegression(max_iter=2000,C=1.0,random_state=SEED).fit(Etr,ytr); R["lr"]=ev(lr.predict_proba(Ete)[:,1])
cmx=confusion_matrix(yte,(pt>=.5).astype(int)); tn,fp,fn,tp=cmx.ravel(); pct=cmx/cmx.sum()*100
lab=np.array([[f"True Negative (TN)\n{tn}\n({pct[0,0]:.1f}%)",f"False Positive
    (FP)\n{fp}\n({pct[0,1]:.1f}%)"],[f"False Negative (FN)\n{fn}\n({pct[1,0]:.1f}%)",f"True Positive
    (TP)\n{tp}\n({pct[1,1]:.1f}%)"]])
plt.figure(figsize=(6.6,5.4));
    sns.heatmap(cmx,annot=lab,fmt="",cmap="Blues",cbar=False,linewidths=2,linecolor="k",xticklabels=["Predicted One-
    time (0)","Predicted Repeat (1)"],yticklabels=["Actual One-time (0)","Actual Repeat
    (1)"],annot_kws={"size":10,"fontweight":"bold"})
plt.title(f"Confusion Matrix on Unseen Test Set (n = {len(yte)})",fontweight="bold"); sv("f9")
plt.figure(figsize=(6.6,5.4)); plt.plot(fpr,tpr,color="#D94A38",lw=2.4,label=f"MLP (AUC = {R['mlp']['auc']:.4f})")
```

11. Appendix: Core Code Architecture (Cont.)

```
fl,tl,_=roc_curve(yte,lr.predict_proba(Ete)[:,1]); plt.plot(fl,tl,color="#2B5F91",lw=1.8,label=f"Logistic Regression
    (AUC = {R['lr']['auc']:.4f})")
plt.plot([0,1],[0,1],"k:",label="Random (AUC = 0.5000)");
    plt.scatter([fpr[J.argmax()]],[tpr[J.argmax()]],color="k",zorder=5,label=f"Youden's J cutoff = {jt:.3f}")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate"); plt.title("ROC Curves on Unseen Test
    Set",fontweight="bold"); plt.legend(loc="lower right",fontsize=8); plt.grid(alpha=.3); sv("f10")
pi=permutation_importance(mlp,Ete,yte,scoring="roc_auc",n_repeats=10,random_state=SEED)
o=np.argsort(-pi.importances_mean)[:10]
R["perm"]=[[Etr.columns[i],round(float(pi.importances_mean[i]),5),round(float(pi.importances_std[i]),5)] for i in o]
json.dump(R,open("results.json","w"),indent=1,default=lambda x: x.item() if hasattr(x,"item") else str(x))
print(json.dumps(R,indent=1,default=lambda x: x.item() if hasattr(x,"item") else str(x)))
```

Academic Declaration: This case study report, exploratory visual artifacts and neural network implementation were developed by Santhosh G (24AD102) for course U21ADP05 at KPR Institute of Engineering and Technology. The dataset is synthetic and all code and figures are reproducible by running the script above.